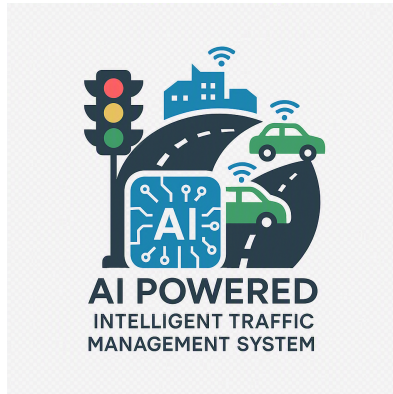




AI-POWERED INTELLIGENT TRAFFIC MANAGEMENT SYSTEM

Smart City AI — Traffic Intelligence Platform

A Project Report

Submitted by	Ashish Kumar Prusty
Programme	B.Tech – Artificial Intelligence & Machine Learning
Institution	GITA Autonomous College, Odisha, India
Project Domain	Computer Vision · Machine Learning · Smart City Systems
Live Deployment	ai-powered-intelligent-traffic-management-system-bkpdj5xodzee.r.streamlit.app

"Making Indian Roads Safer through Artificial Intelligence — reducing accidents, easing congestion, and helping build smarter cities."

Declaration

I hereby declare that the project titled **“AI-Powered Intelligent Traffic Management System”** is an original piece of work carried out by me, under my own initiative, as part of my B.Tech programme in Artificial Intelligence & Machine Learning. The system design, dataset exploration, application development, and deployment described in this report reflect work that I personally planned, built, tested, and iterated on. All datasets, libraries, and open-source tools used are acknowledged in the References section.

Ashish Kumar Prusty

B.Tech – Artificial Intelligence & Machine Learning

GITA Autonomous College, Odisha

Acknowledgement

I would like to express my sincere gratitude to the faculty of the Department of Artificial Intelligence & Machine Learning at GITA Autonomous College for their guidance and support throughout the course of this project. I am also grateful to the open-source community behind Ultralytics YOLOv8, Streamlit, OpenCV, Plotly, and the UA-DETRAC dataset, whose freely available tools and data made this project possible. Finally, I thank my peers and family for their continuous encouragement during the development and deployment of this system.

Abstract

India accounts for one of the highest numbers of road traffic accidents in the world, with a large share attributed directly to poor traffic management, signal inefficiency, and congestion rather than vehicle failure alone. This project, **Smart City AI**, presents an AI-powered Intelligent Traffic Management System designed to address this problem through automated, camera-based traffic analysis. The system uses a YOLOv8 deep learning model to detect and classify vehicles from an uploaded traffic image or video, then automatically pipelines that detection through traffic analytics (density, congestion, and waiting-time estimation), a forecasting stage that predicts near-future traffic conditions, a live monitoring dashboard, and a downloadable AI-generated report — all without requiring the user to re-upload data at each stage. The application is built as a single-page Streamlit platform with a shared session state architecture, is deployed publicly on Streamlit Community Cloud, and is accessible as a live web application. The project was developed in structured phases — dataset acquisition and exploratory data analysis on the UA-DETRAC vehicle-detection dataset, model integration, full-stack web application development, and cloud deployment — and is designed with a clear roadmap toward becoming a real-world, government-usable smart-traffic platform, incorporating planned future modules such as automatic number-plate recognition within legal data-privacy boundaries, adaptive traffic-signal timing, and live CCTV integration.

Table of Contents

1. Introduction

1.1 Background and Motivation

1.2 Problem Statement

1.3 Objectives

1.4 Scope of the Project

2. Existing Systems and Motivation for This Approach

3. System Analysis and Architecture

3.1 Proposed System

3.2 System Architecture

3.3 Technology Stack

4. Dataset and Exploratory Data Analysis

5. Methodology

5.1 Vehicle Detection (YOLOv8)

5.2 Traffic Analytics

5.3 Traffic Prediction

5.4 Live Monitoring

5.5 Automated Workflow Pipeline

6. Implementation

7. Results and Discussion

8. Testing

9. Future Scope

10. Conclusion

11. References

1. Introduction

1.1 Background and Motivation

Road traffic accidents remain one of India's most pressing public safety concerns. A large proportion of these accidents — commonly cited at around 75–85% in traffic-safety discussions — are linked not to a single catastrophic failure, but to everyday traffic conditions: congestion, poor signal timing, and a lack of real-time visibility into how traffic is actually behaving at a given junction. Most traffic signals in Indian cities still operate on fixed timers that do not respond to actual traffic volume, meaning a nearly empty road can hold a green light as long as a heavily congested one. This mismatch between real conditions and signal behaviour is a solvable, data-driven problem — and it is the core motivation behind this project.

1.2 Problem Statement

There is a need for an accessible, AI-driven system that can look at a traffic scene (from an uploaded image, video, or eventually a live camera feed), automatically understand how many vehicles are present and of what type, quantify how congested that scene is, forecast how conditions are likely to change in the near future, and present all of this in a single, easy-to-use dashboard — without requiring specialised hardware or manual data entry.

1.3 Objectives

- Design and build an end-to-end AI pipeline that detects and classifies vehicles from traffic imagery.
- Compute meaningful traffic analytics (density, congestion level, estimated waiting time) from detection results.
- Build a forecasting module that predicts near-future traffic conditions from current detections and context (time of day, weather, holiday status).
- Provide a live monitoring view and a downloadable AI-generated report of every analysis run.
- Package the entire system as a single, automatically-guided web workflow that a non-technical user can operate.
- Deploy the system as a publicly accessible live web application.

1.4 Scope of the Project

The current implementation focuses on image- and video-based traffic analysis uploaded directly by the user, using a general-purpose YOLOv8 object detector for vehicle classes, combined with a rule-based analytics layer and a hybrid (model-assisted, heuristic-backed) prediction layer. It does not, in its current form, connect to live government CCTV infrastructure, control physical traffic signals, or access private RTO ownership records — these are explicitly identified as future-phase integrations in Section 9, contingent on institutional partnership and data-access authorization.

2. Existing Systems and Motivation for This Approach

Most traffic signal infrastructure currently deployed across Indian cities relies on fixed-cycle timers, sometimes supplemented by manual traffic-police control during peak hours. Where camera-based systems do exist, they are typically closed, expensive, city-scale deployments (often vendor-specific ANPR or violation-detection systems) that are not accessible for students, researchers, or smaller municipal bodies to inspect, extend, or pilot cheaply. This creates a gap: there is limited tooling that is simultaneously (a) built on modern, accurate deep-learning detection, (b) transparent and inspectable rather than a black box, and (c) deployable as a working, testable prototype without large infrastructure investment. Smart City AI is designed to sit in that gap — a fully open, understandable pipeline that can be demonstrated end-to-end on a laptop or a free cloud tier, while being architected so that each stage (detection, analytics, prediction, monitoring) could later be swapped for a production-grade, hardware-integrated equivalent without redesigning the system.

3. System Analysis and Architecture

3.1 Proposed System

The proposed system is a single, continuous AI workflow rather than a set of disconnected tools. A user uploads one image or video on the Home page and starts the pipeline; the system then automatically walks through Vehicle Detection, Traffic Analytics, Traffic Prediction, and Live Monitoring in sequence, storing every result in a shared session state, before returning to the Home page with a complete, downloadable report. At any point after the initial run, the user can freely revisit any individual stage without repeating the earlier steps.

3.2 System Architecture

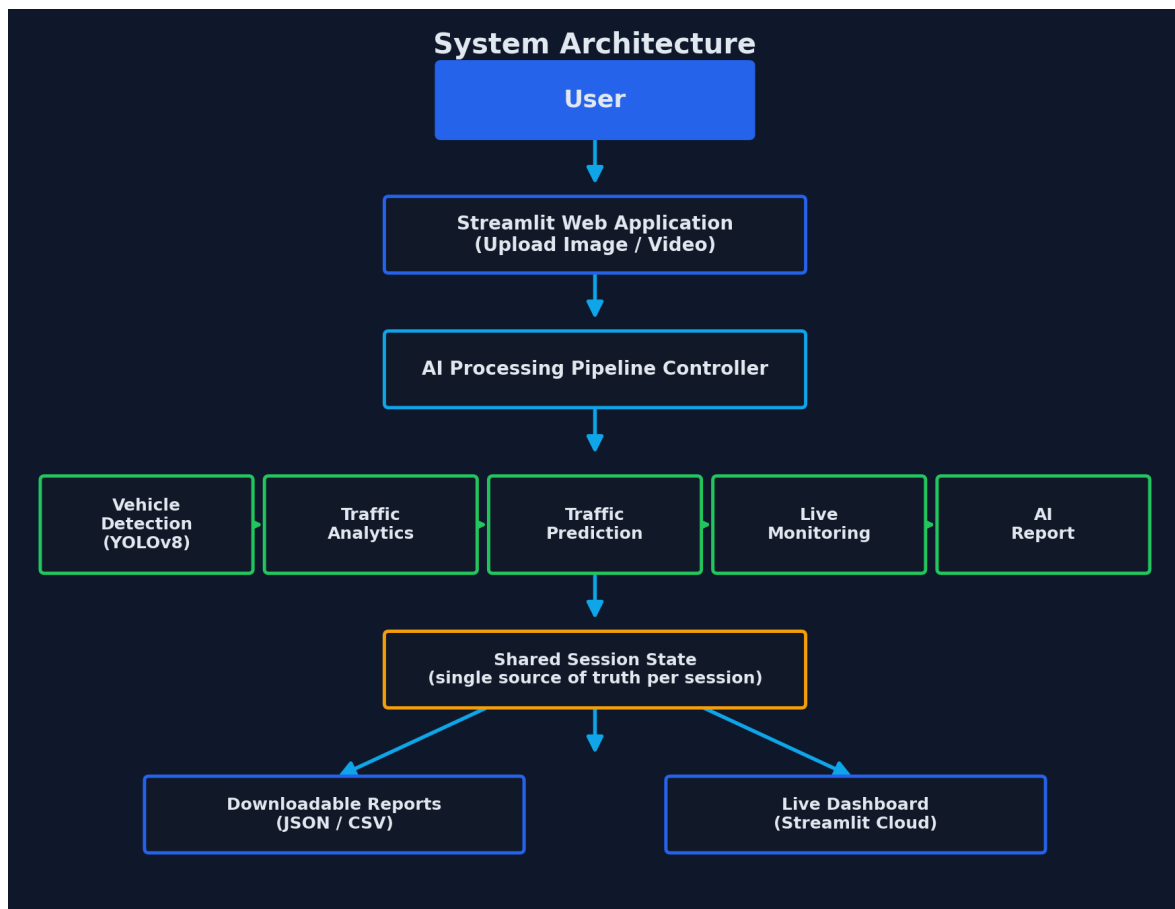


Figure 3.1 — High-level system architecture, from user upload through the AI pipeline to the shared session state and output layer.

3.3 Technology Stack

Layer	Technology
Programming Language	Python 3.11
Object Detection	YOLOv8 (Ultralytics)
Computer Vision	OpenCV

Web Application / Dashboard	Streamlit
Data Visualization	Plotly (Express & Graph Objects)
Data Handling	Pandas, NumPy
Deployment	Streamlit Community Cloud
Version Control	Git & GitHub

4. Dataset and Exploratory Data Analysis

The foundation-phase development and detection model validation for this project used the **UA-DETRAC** dataset, a widely used vehicle-detection benchmark, obtained in a YOLO-annotated format with images and labels pre-split into training and validation sets. Before any model training or inference logic was written, a structured, three-notebook exploratory data analysis (EDA) phase was carried out:

- **Dataset Exploration** — validated folder structure, confirmed image/label counts matched, checked for missing files, and inspected the dataset's *data.yaml* configuration.
- **Dataset Visualization** — rendered random samples with YOLO bounding boxes overlaid, checked resolution consistency, and grid-reviewed multiple annotated images at once to visually confirm label correctness.
- **Annotation Analysis** — computed class distribution, objects-per-image statistics, and bounding-box width/height/area/aspect-ratio distributions to understand dataset balance and object scale before training.

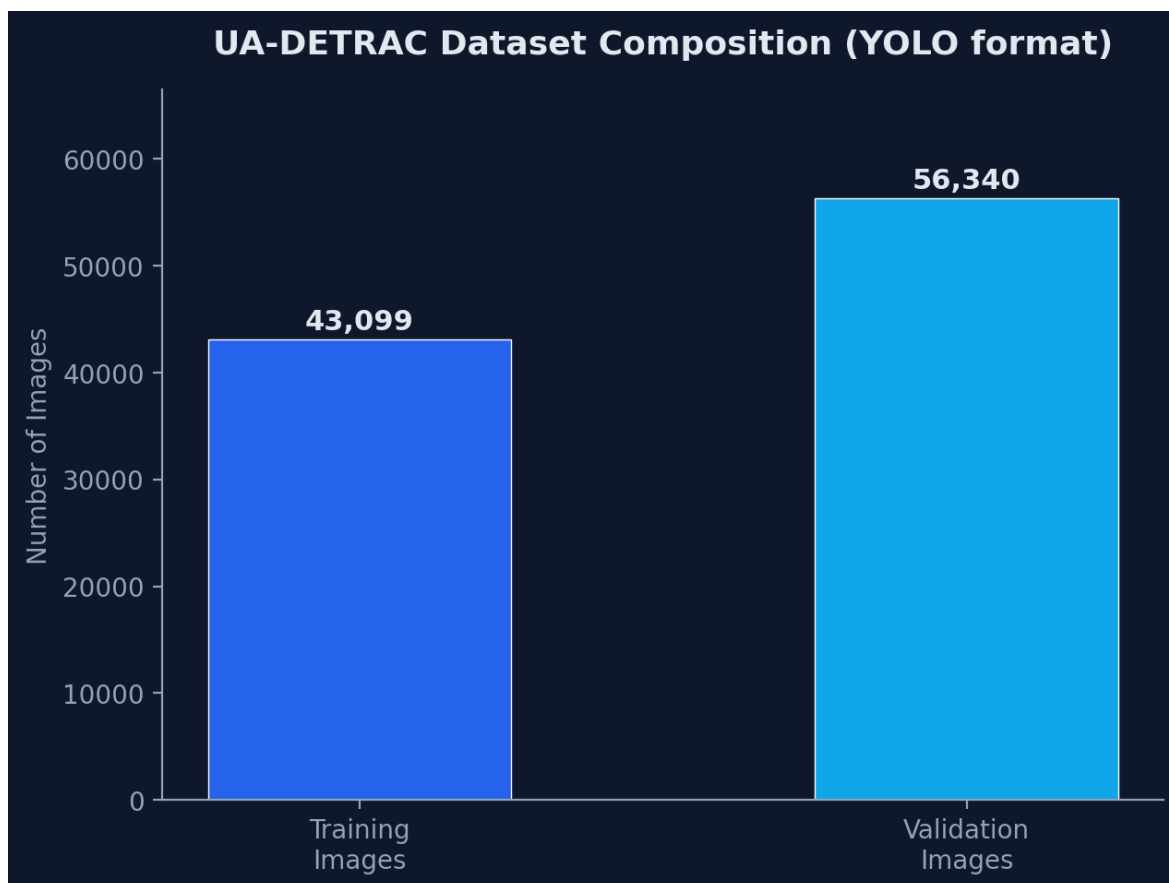


Figure 4.1 — UA-DETRAC dataset composition used during the data-understanding phase.

5. Methodology

5.1 Vehicle Detection (YOLOv8)

Vehicle detection is performed using a YOLOv8 model restricted to vehicle-relevant object classes (car, motorcycle, bus, truck). Given an uploaded image, the model returns bounding boxes, class labels, and confidence scores for every detected vehicle; these are drawn directly onto the image and the per-class counts are extracted at detection time and stored centrally, so every later stage of the pipeline works from the same, verified count rather than re-deriving it.

5.2 Traffic Analytics

From the raw vehicle count, the analytics layer derives a traffic-density label (Low / Medium / High), a 0–100% congestion score, and an estimated average waiting time. These are visualized through a vehicle-distribution bar and pie chart, a congestion gauge, a traffic-trend line chart built from the session's detection history, and a road-zone density heatmap.

5.3 Traffic Prediction

The prediction stage answers a fundamentally different question from detection and analytics: not “what is on the road right now”, but “what is traffic likely to look like shortly from now, given current conditions.” The current vehicle count is combined with user-selectable context — hour of day, weather condition, and holiday status — to forecast a future vehicle count, predicted congestion, a recommended traffic-signal green-time adjustment, and a model confidence score. This stage is designed to opportunistically use a trained forecasting model where available, with a transparent, explainable heuristic fallback so the feature is always functional.

5.4 Live Monitoring

The monitoring stage consolidates detection, analytics, and prediction results into a single live-style dashboard — showing current vehicle counts, a real-time trend chart, system-module status, and AI-generated textual insight about the current congestion state.

5.5 Automated Workflow Pipeline

All four stages above are chained into one automatic pipeline: starting it from the Home page walks the user through Detection, Analytics, Prediction, and Monitoring in order, generates a final report, and returns to Home with a complete summary and download options — while every stage also remains independently accessible for standalone exploration afterward.

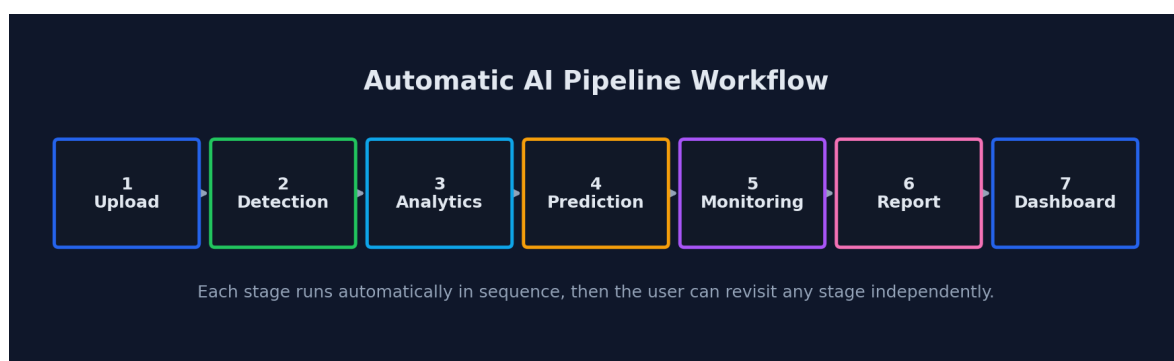


Figure 5.1 — The automatic, end-to-end AI pipeline workflow.

6. Implementation

The production application is implemented as a single, self-contained Streamlit script, deliberately kept dependency-light at the entry-point level so that deployment failures in one module cannot silently break the whole app. Supporting research and development work — dataset exploration notebooks, a modular *traffic_ai* package covering detection, tracking, counting, analytics, optimization, and database concerns, and a FastAPI backend/React frontend scaffold for a future full multi-service architecture — is organized in a structured project repository.

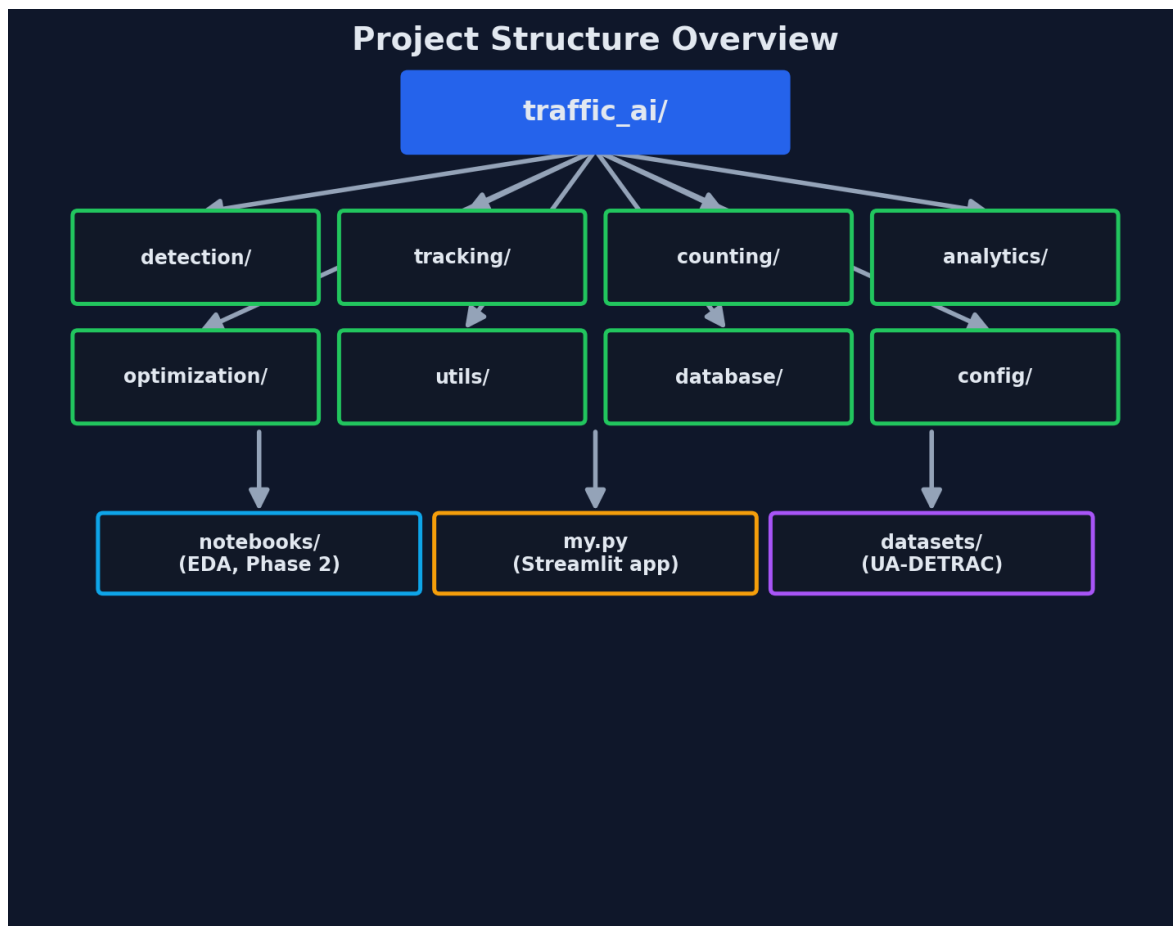


Figure 6.1 — Project repository structure.

Deployment

The application is deployed on Streamlit Community Cloud, connected directly to the project's GitHub repository. During deployment, dependency resolution failures were traced to the cloud environment defaulting to a newer Python version than several dependencies had pre-built wheels for; this was resolved by explicitly pinning the runtime Python version and relaxing dependency version constraints to compatible ranges, after which the build succeeded and the application went live at:

ai-powered-intelligent-traffic-management-system-bkpdj5xodzeer.streamlit.app

7. Results and Discussion

The deployed system was validated by uploading real traffic imagery and confirming correct end-to-end behaviour across every pipeline stage. The detection engine reliably identifies multiple vehicles per frame with associated confidence scores, as shown below on an actual sample processed by the live application.

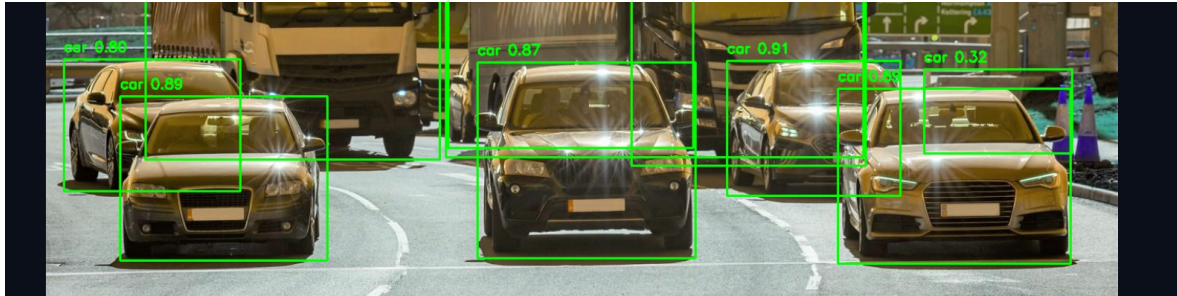


Figure 7.1 — Sample YOLOv8 vehicle detection output from the deployed application, showing bounding boxes and per-vehicle confidence scores.

Beyond raw detection accuracy, the key result of this project is architectural: a single upload now reliably flows through analytics, prediction, and monitoring without the user re-entering data at each step, and every stage remains independently revisitable. This was validated by confirming that vehicle counts, density labels, and congestion scores stay consistent across Detection, Analytics, and Monitoring for the same uploaded input, and that the Prediction stage correctly produces a distinct, forward-looking forecast rather than repeating the current count.

8. Testing

Testing was carried out at two levels. At the data level, the three exploratory-analysis notebooks served as automated validation of the training dataset itself — confirming that every image has a matching label file, that no images or labels are missing, and that annotation coordinates are well-formed before any model work began. At the application level, each pipeline stage (Detection, Analytics, Prediction, Monitoring, Report) was manually tested both as part of the automatic end-to-end pipeline and as a standalone page, across both image and video uploads, with particular attention to session-state consistency — ensuring a value computed once (such as vehicle count) is correctly reflected on every page that depends on it, rather than resetting or diverging between pages.

9. Future Scope

This project is intentionally architected as the foundation of a larger platform rather than a finished, closed system. The following phases outline a realistic path from the current working prototype toward a genuinely government-deployable smart-traffic platform.

Phase 2 — Recognition and Compliance Features

- Automatic Number Plate Recognition (ANPR) integrated with detection results.
- Vehicle information lookup restricted strictly to publicly available RTO/Parivahan data (registration class, RTO, fuel type, insurance and PUC validity) — explicitly excluding any private data such as owner phone number, address, or financial details, in line with data-privacy law.
- Traffic-rule violation detection (helmet, wrong-lane, red-light crossing).

Phase 3 — Live Infrastructure Integration

- Live CCTV / RTSP camera feed support in place of upload-only input.
- Multi-junction support with persistent historical storage, replacing the current single-session data model.
- Vehicle speed estimation and lane-level analysis via multi-object tracking.
- Simulated adaptive traffic-signal timing, validated using a traffic simulator (such as SUMO) before any real controller integration is attempted.
- An emergency-vehicle detection and green-corridor recommendation module.

Phase 4 — Real-World Deployment Path

Adoption by an actual municipal or traffic-police body realistically requires a pilot partnership — running the system against footage from one real junction, in coordination with a local authority, to validate accuracy and build the trust needed for any larger rollout. This project is deliberately positioned as a decision-support and monitoring layer that could integrate with existing signal-control and CCTV infrastructure through formal API access, rather than assuming direct control over physical traffic hardware from day one.

10. Conclusion

This project demonstrates that a modern, deep-learning-based traffic analysis system — covering detection, analytics, prediction, and monitoring — can be designed, built, and deployed as a single, coherent, publicly accessible platform without requiring specialized hardware or institutional infrastructure. Starting from disciplined dataset exploration and progressing through a self-contained YOLOv8 detection engine, a shared-state web application, and a live cloud deployment, Smart City AI shows a concrete, working path toward the stated mission: using artificial intelligence to make Indian roads safer by giving traffic conditions the visibility they need to be managed proactively rather than reactively. The system, as it stands, is both a complete academic deliverable and a credible starting point for the government-grade roadmap outlined in Section 9.

11. References

- Jocher, G., Chaurasia, A., Qiu, J. (2023). *Ultralytics YOLOv8*. <https://github.com/ultralytics/ultralytics>
- Wen, L. et al. *UA-DETRAC: A New Benchmark and Protocol for Multi-Object Detection and Tracking*.
- Streamlit Inc. *Streamlit Documentation*. <https://docs.streamlit.io>
- Bradski, G. (2000). *The OpenCV Library*. Dr. Dobb's Journal of Software Tools.
- Plotly Technologies Inc. *Plotly Python Open Source Graphing Library*. <https://plotly.com/python/>
- Ministry of Road Transport and Highways, Government of India. *Road Accidents in India* (annual report series).
- Parivahan Sewa, Government of India. <https://parivahan.gov.in> — official vehicle registration information portal referenced for the planned ANPR/lookup feature's legal data-access boundary.